

Tide — Technical Design Document

Test Integrity in Developer Environments — System architecture, detection strategies, and decision algorithms

Hackathon build · Branch **rebuild** · Status: built and tested (114 automated tests, single-device rehearsal against the live Jev API)

Contents

- 1. System overview and component roles
- 2. Why not polling — real-time transport design
- 3. Exam lifecycle state machine
- 4. Detection layer: watchers, sampling rates, and algorithms
- 5. Code-reuse detection — fingerprinting and winnowing
- 6. Decision pipeline — rules, cache, AI classifier, heuristics
- 7. Enforcement mechanics
- 8. Clock synchronization strategy
- 9. Data model and evidence snapshots
- 10. Security model and threat table

1. System overview and component roles

Tide is a three-process system: a lightweight agent on every student PC, a central server on the teacher's laptop, and a browser console the teacher watches. The design principle is **"watch the real desktop, don't replace it"** — students keep using VS Code, Wireshark, VMware; Tide observes OS-level signals and intervenes only on specific, well-defined violations.

Component	Stack	Responsibility
Tide Agent	Python 3.12 + pywin32, psutil, uiautomation, mss, pywebview (packaged via PyInstaller)	Runs on every student PC. Watches OS signals, applies local rules instantly, collects evidence
Tide Server	Python 3.12, FastAPI, WebSockets, SQLModel/SQLite	Runs on the teacher laptop. Pairs seats, runs the classification pipeline, makes decisions
Teacher Console	React + Vite + TypeScript	Served by the server. Live seat grid, flag feed, per-student timeline, results/export

Why not Electron: anti-cheat needs OS-level signals (foreground window, process list, browser address bar, clipboard, USB) that pywin32/uiautomation expose in a few lines; Node would need fragile native add-ons. The Python agent is ~25 MB vs. Electron's ~150 MB. The agent's small UI (join screen, timer pill, block overlay) uses **pywebview**, rendering HTML through the Edge WebView2 already built into Windows — so it gets a web-quality UI with no bundled Chromium.

2. Why not polling — real-time transport design

Tide uses **persistent WebSocket connections**, not HTTP polling, for anything time-sensitive. One socket per seat (`ws://<server>:8765/ws/agent`) and one for the console (`ws://<server>:8765/ws/console`). Polling was rejected because enforcement (closing a ChatGPT tab, killing a process) needs to happen within ~1-2 seconds of detection, and 60 seats polling on an interval either lags badly or floods the server — a push model scales to a full lab on one laptop with no extra load at idle.

2.1 Message envelope

Every message is JSON with a single discriminator field `t`. The first message on any socket must be `hello` with the seat's auth token — anything else closes the socket immediately (code 4401).

Direction	Type (t)	Purpose
Agent → Server	hello	Auth handshake: token, agent_version, local_time
Agent → Server	heartbeat	Liveness ping every 3s, carries current foreground app
Agent → Server	preflight	Internet status, extensions found, apps auto-closed, file count
Agent → Server	event	Allowed activity — timeline only, no decision needed
Agent → Server	signal	Ambiguous activity — enters the classification pipeline
Agent → Server	flag	A local rule already fired: kind, severity, action taken
Agent → Server	evidence	Screenshot (base64 JPEG) attached to a flag
Agent → Server	snapshot	Changed exam files only, every 30s
Server → Agent	welcome	Seat number, roll, question set, policy, server clock, ends_at

Direction	Type (t)	Purpose
Server → Agent	start	Question files (base64) + exam end time
Server → Agent	act	Command: close_tab / kill / overlay, with target + reason
Server → Agent	time	Updated ends_at after a teacher extension
Server → Agent	notice	Broadcast text message from teacher
Server → Agent	end	Exam over — collect files and submit now

Offline resilience: if an agent loses the connection to the server (network switch, Wi-Fi drop), local rules keep enforcing without a round-trip, and events are buffered to disk and flushed on reconnect. The server independently flags the connectivity gap so it's never silently lost.

3. Exam lifecycle state machine

SETUP → LOBBY/PRE-FLIGHT → LIVE → SUBMIT → REVIEW. Each stage exists to close a specific cheating vector rather than as generic workflow structure.

Stage	What happens	Cheat vector closed
Setup	Teacher uploads Set A / Set B files, picks an app policy preset, sequences the test, and changes the join code.	Questions are distributed to the classroom.
Lobby / Pre-flight	Agent discovers the server via UDP broadcast, pairs with join code, confirms and clears the 25-state seat token.	Confirms and clears the 25-state seat token.
Live / Start	Teacher presses Start. Server pushes Set A to odd seats, Set B to even seats, and the class begins.	Questions are served to the classroom.
Submit	Student presses Submit, or time runs out. Agent zips the exam folder and uploads it to the server.	NA and uploads the seat goals, reviews stop, agent e-
Review	Teacher sees per-student timeline, flags with screenshots, code-generated feedback across submission, and all pairs.	Evidence is backed across submission, and all pairs.

3.1 Internet policy — a per-exam setting, not a fixed rule

Each exam picks one of two modes. This was a deliberate pivot from an earlier "always offline" design.

Mode	Default	Behavior
Allowed · monitored	Yes	Students stay online. Every site/app is checked; deny-listed sites close instantly, unknown
Blocked	Opt-in, strict labs	Questions are withheld until the PC is confirmed offline. Going online mid-exam turns the

Rationale: forcing a lab fully offline is exactly what fails today — students re-plug a cable, use a hotspot, or grab answers before disconnecting. Strong monitoring makes leaving the internet on *safe*, and offline-only cheats (local AI apps, old files, USB) are caught identically in both modes.

4. Detection layer: watchers, sampling rates, and algorithms

Every watcher on the agent emits a uniform {kind, data, ts} signal. Sampling intervals are tuned per-signal: fast enough to catch a cheat within a couple of seconds, slow enough not to burn CPU on 60 machines.

Watcher	Interval	Mechanism	What it catches
Foreground window + browser URL	500 ms	GetForegroundWindow → PID → process name (psutil). If it's a browser, it checks the URL.	ChatGPT, Claude, Gemini, Copilot web, perplexity, A
Processes + extensions	2 s (ext: 60 s)	psutil.process_iter diffed against last scan; matches on file	Receipts, Abigail, file name, cross RLS, Copilot, Coda
Network	probe 5s adapters 2s	psutil.net_if_stats for new adapters (Wi-Fi, USB tether, hotspot)	De-enabled Wi-Fi, ability to spoof HTTP GET to Nipos
Files / USB / clipboard	clipboard 1s, USB on insert	psutil.disk_partitions diff for new removable drives. Clipboard	USB drives, large pastes, images, streams, code
Code bursts	per 30s snapshot	Server-side: compares two successive 30-second file snapshots	Bulk-pasted solutions that trip up address flagging, Dye

4.1 Evidence capture

On any flag of severity $\geq$ medium, the agent takes a screenshot (mss, primary monitor, downscaled to 1280px JPEG, ~120 KB) and attaches it to that flag record. This turns every accusation into something a teacher can visually verify rather than trusting an automated label blindly.

5. Code-reuse detection — fingerprinting and winnowing

During pre-flight, the agent walks the student's user folders and non-system drives for source/doc files (.c .cpp .h .py .java .js .ts .sql .txt .md .pdf .docx .ipynb), capped by count and size. For code files it does not store the raw text — it stores a **normalized fingerprint**:

- **Normalize** — strip whitespace and comments so formatting differences don't matter.
- **Tokenize into n-grams** — break the normalized text into overlapping windows of tokens.
- **Winnowing hash** — hash each n-gram, then keep only the minimum hash within each sliding window (the winnowing algorithm, the same family of technique used by MOSS-style plagiarism detectors). This gives a compact, robust signature that survives minor edits and reordering.

At exam time, two checks run against this inventory: opening a window whose title matches an inventoried filename raises **"Pre-exam file opened"**; a new or changed file inside `C:\Exam\<roll>\` whose fingerprint overlap with an inventoried file crosses a threshold raises **"Old code reused"**, reported with a similarity percentage and the source path. The same fingerprinting also powers cross-student similarity pairs in the Results view, for copy-from-neighbor detection.

6. Decision pipeline — rules, cache, AI classifier, heuristics

This is the core strategic design of Tide: act instantly on the obvious, ask an AI about the ambiguous, and never let an AI call auto-punish anyone without a high confidence bar. Every signal flows through up to five stages, short-circuiting as soon as one produces a verdict:

```
signal → (A) agent-local rules → (B) server rules → (C) verdict cache → (D) Jev AI classifier → (E) offline keyword heuristics (fallback only)
```

6.1 (A) Local rules — instant, offline, zero round-trip

The policy (allow list + deny list + domain deny list) is pushed to the agent once, at pairing. Matches here act with no network call at all: deny-listed browser host → close tab; deny-listed process → kill; internet reachable in blocked mode → overlay until offline; USB inserted → flag only.

6.2 (D) Jev — the AI classifier for unknowns

Jev (TypeSafe AI, via OpenRouter's Decisions API, model `~typesafe/jev-latest`) is deliberately **not** a chat LLM. It takes typed, fixed-choice questions and returns a label + calibrated probability — no free text to parse or guard against. That property is what makes it safe to threshold and auto-act on. Measured latency: ~0.4s per call at fractions of a cent, which is what makes classifying every ambiguous event from 60 seats economically viable.

Per signal, Tide sends only the minimum needed context — exam's allowed-app policy plus the event (process, window title, host) — and asks two typed questions:

Question	Type	Options / range
activity	single choice	allowed_tool, ai_assistant, communication, web_lookup, remote_or_file_share, other
violation	probability	"Is this against the exam policy?" — 0.0 to 1.0

Thresholds — the auto-act bar is set high on purpose:

Condition	Result
activity ∈ {ai_assistant, communication, remote_or_file_share} AND confidence ≥ 0.90	Auto-act (close tab / kill) + critical flag
violation ≥ 0.60	Flag for teacher review (amber) — no automatic action
otherwise	Log only, visible in the timeline

Verdicts are cached in-memory per (browser, host) or (process, title-pattern), so the same window is classified by Jev only once per exam, not once per occurrence. Only Jev-produced verdicts are cached, so a Jev outage self-heals as soon as the key/connection is restored.

Real measurements from the demo build: Chrome/poe.com "Fast AI Chat" → ai_assistant, p=0.97 → auto-closed. notegpt.exe → ai_assistant, p=0.99 → auto-killed. Edge/geeksforgeeks.org → web_lookup, p=0.97 → flagged for review, not closed. VS Code editing main.c → allowed_tool, p=1.00 → nothing.

6.3 (E) Offline heuristics — the fail-safe fallback

If no OpenRouter key is configured, or Jev is unreachable, the server falls back to plain regex keyword matching (e.g. "chatgpt|claude|copilot" for ai_assistant, "whatsapp|discord|slack" for communication). Confidence from this path is hard-capped at 0.70 — below the 0.90 auto-act threshold — so a degraded classifier can only ever flag for review, never auto-punish. The console header shows this fallback is active. This is a deliberate reliability design: **the demo — and a real exam — never breaks because of an API outage.**

6.4 Severity → seat color

Severity	Examples	Seat tile
Critical	Auto-acted AI site/app, agent killed, internet reachable in blocked mode	Red
High	USB inserted, old code reused, clipboard match, Jev violation ≥ 0.8	Red
Medium	Code burst, LAN peer connection, AI extension installed, Jev 0.6–0.8	Amber
Info	App switches, allowed tools, routine snapshots	Green (timeline only)
—	No heartbeat for 10s	Grey — "Agent offline"

A seat's color is always its worst currently-open flag; dismissing that flag lets the color drop.

7. Enforcement mechanics

- **Close tab:** bring the browser window to the foreground, send Ctrl+W via SendInput, re-read the address bar 300ms later — if the AI host is still present, escalate to killing the whole browser process.
- **Kill app:** psutil terminate, hard-kill after a 1-second grace period.
- **Block overlay:** a full-screen, always-on-top red window with a single line ("ChatGPT blocked — reported"), auto-dismissing after 4 seconds. For the internet-blocked case, the overlay persists until the connectivity probe fails again.
- Every action and its result ("Auto-closed", timestamp) is written back onto the flag record, so the teacher console always shows what actually happened, not just what was attempted.

8. Clock synchronization strategy

The server is the single source of truth for exam time — agents never run an independent countdown that could drift or be tampered with by changing the local PC clock.

- At pairing, the agent measures its clock offset as (server_now – local_now), corrected for half the round-trip time of the pairing request.
- Start sends one absolute timestamp, ends_at — not a duration — so the agent's local countdown is just "ends_at minus corrected local time", immune to drift.
- A teacher extension pushes a new ends_at instantly to one seat or all.
- A restarted or reconnected agent receives the current ends_at in its next welcome message — restart-safe by construction.

9. Data model and evidence snapshots

State is stored in SQLite (SQLModel), sized for a single lab (60 seats) on the teacher's laptop:

```
exam(id, title, duration_s, join_code, state, started_at, ends_at, policy_json) exam_file(id,
exam_id, set_name, name, data) seat(id, exam_id, seat_no, roll, hostname, token_hash, set_name,
state, preflight_json, last_seen, ends_at_override, fg_app, simulated) event(id, seat_id, ts, kind,
data_json) -- the timeline flag(id, seat_id, ts, kind, severity, title, source, label, confidence,
data_json, action, screenshot, status, reviewed_at, ref) snapshot(id, seat_id, ts, path, sha, text,
line_count) submission(id, seat_id, ts, file_name, auto)
```

Snapshotting strategy: every 30 seconds the agent diffs the exam folder and sends only changed files (by content hash) — never the whole folder repeatedly. This keeps bandwidth flat regardless of file size and gives the server a time-series of code growth per student, which both powers the code-growth chart and feeds the 40-line/30s burst-detection rule in §4.

The Jev verdict cache is intentionally in-memory only, scoped to one exam per server run — it does not need to survive a server restart, since a restart mid-exam is rare and a cold cache just re-classifies the next occurrence of each unique window.

10. Security model and threat table

Threat	Demo mitigation	Production hardening
Fake agent / forged flags for a class	Re-seat random 256-bit token issued at pairing; requires a valid WebSockets message signature	Telemetry with binary integrity checks
Student kills the agent process	Heartbeat stops → seat turns grey + critical flag with Agent ID	Agent runs as a Windows service (SYSTEM); students aren't admin
Student never starts the agent	Seat missing from the grid; teacher sees e.g. "59/60"	Agent auto-starts at login via the service; hostname roster shows
Tampering with local rules	Rules are pushed from the server at pairing; agent code is compiled	Binary integrity checks in the hello handshake
Changing the PC's local clock	Server clock is authoritative (\$8) — local clock changes	Seems to have no effect
Unauthorized console access	Teacher PIN; console only served on the teacher's laptop	Overlaid prompts, per-lab isolation
Privacy of collected data	Agent runs only during the exam and exits after submission	For, secretions policy, change at meta-level only, window/app metadata

Honest scope note: this document describes the demo build on branch rebuild (Windows-only, single teacher laptop, 59 seats simulated alongside 1 real seat). Production hardening items above are pitched, not built.